Universidade Federal do ABC

16 1 Anthony Deleon de Almeida Costa AD

Situação	Finalizada
Iniciado	sábado, 21 fev. 2026, 10:58
Concluído	sábado, 21 fev. 2026, 11:07
Duração	9 minutos 2 segundos
Nota	**10,00** de um máximo de 10,00(**100**%)

Questão 1 | Correto Atingiu 2,00 de 2,00

Algoritmo 1

Entrada: `A[1...n], n`

```
B ← novo array com n posições
Para i de 1 até n faça
    B[i] ← A[n-i+1]
A ← B
Retornar A
```

O que este algoritmo faz?

Inverte o array A usando um array auxiliar B

Qual é o tempo de execução deste algoritmo em notação Θ?

- $\Theta(1)$
- $\Theta(\log(n))$
- $\Theta(n)$
- $\Theta(n \log n)$
- $\Theta(n^2)$

Atingiu 1,00 de 1,00

Questão 2 | Correto Atingiu 2,00 de 2,00

Algoritmo 2

Entrada: `A[1...n], n`

```
Para i de 1 até ⌊n/2⌋ faça
    temp ← A[i]
    A[i] ← A[n-i+1]
    A[n-i+1] ← temp
Retornar A
```

O que este algoritmo faz?

Inverte o array A sem array auxiliar

Qual é o tempo de execução deste algoritmo em notação Θ?

- \(\Theta(1)\)
- \(\Theta(n)\)
- \(\Theta(\log n)\)
- \(\Theta(n \log n)\)
- \(\Theta(n^2)\)

Atingiu 1,00 de 1,00

Ambos os algoritmos possuem o mesmo tempo de execução assintótico.

A principal diferença entre as soluções está no uso de memória:

- O Algoritmo 1 cria um novo array auxiliar B de tamanho n para armazenar os elementos invertidos e depois copia esse array de volta para A, utilizando \(\Theta(n)\) de espaço adicional.
- O Algoritmo 2 é uma solução **in-place**, o que significa que ele inverte o array diretamente no próprio espaço de memória original, usando apenas uma variável temporária.

Soluções in-place são mais eficientes em termos de memória, pois utilizam apenas \(O(1)\) de espaço extra, independentemente do tamanho da entrada. Podem ser mais difíceis de programar.

Questão 3 | Correto Atingiu 2,00 de 2,00

Considere o problema de verificar se existem elementos duplicados em um array A[1...n] de inteiros entre 1 e 100n.

Algoritmo 1

Entrada: `A[1...n], n`

```
Para i de 1 até n-1 faça
    Para j de i+1 até n faça
        Se A[i] = A[j] então
            Retornar Verdadeiro
Retornar Falso
```

Qual é o tempo de execução do Algoritmo 1?

- \(O(n)\)
- \(O(n \log n)\)
- \(O(n^2)\) ✓
- \(O(n^3)\)

Atingiu 1,00 de 1,00

Qual é o espaço adicional usado pelo Algoritmo 1?

- \(O(1)\) ✓
- \(O(n)\)
- \(O(n \log n)\)
- \(O(n^2)\)

Atingiu 1,00 de 1,00

Questão 4 | Correto Atingiu 2,00 de 2,00

Algoritmo 2

Entrada: `A[1...n], n`

```
Heapsort(A)
Para i de 1 até n-1 faça
    Se A[i] = A[i+1] então
        Retornar Verdadeiro
Retornar Falso
```

Obs: Heapsort tem tempo de execução \(O(n \log n)\) e usa espaço adicional \(O(1)\).

Qual é o tempo de execução do Algoritmo 1?

- \(O(n)\)
- \(O(n \log n)\) ✓
- \(O(n^2)\)
- \(O(n^3)\)

Atingiu 1,00 de 1,00

Qual é o espaço adicional usado pelo Algoritmo 1?

- \(O(1)\) ✓
- \(O(n)\)
- \(O(n \log n)\)
- \(O(n^2)\)

Atingiu 1,00 de 1,00

Questão 5 | Correto Atingiu 2,00 de 2,00

Algoritmo 3

Entrada: `A[1...n], n`

```
C ← novo array com 100n posições inicializadas com 0
Para i de 1 até n faça
    Se C[A[i]] = 1 então
        Retornar Verdadeiro
    C[A[i]] ← 1
Retornar Falso
```

Qual é o tempo de execução do Algoritmo 1?

- \(O(n)\) ✓
- \(O(n \log n)\)
- \(O(n^2)\)
- \(O(n^3)\)

Atingiu 1,00 de 1,00

Qual é o espaço adicional usado pelo Algoritmo 1?

- \(O(1)\)
- \(O(n)\) ✓
- \(O(n \log n)\)
- \(O(n^2)\)

Atingiu 1,00 de 1,00

« Lista 3A Voltar para Curso Slides 4 »